

PKCERT AI & Software Development Internship

Task 26 – Introduction to Transformers & the Attention Mechanism (Conceptual)

Objective:

Build a rigorous conceptual and mathematical understanding of the attention mechanism and the Transformer architecture. Interns will derive the scaled dot-product attention formulation from first principles, reason precisely about multi-head attention, positional encoding, and the encoder-decoder structure, and critically analyze why Transformers superseded recurrent architectures for sequence modeling. This task is explicitly conceptual: no attention mechanism or Transformer block is to be implemented from scratch. All applied work must use existing pretrained models and established libraries.

Total Marks: 100

Part A – From Recurrence to Attention (20 Marks)

- Summarize, with reference to the RNN/LSTM derivations from Task 25, the specific structural bottlenecks of recurrent architectures (sequential computation, long-range dependency decay, fixed-size context propagation) that motivated the development of attention-based mechanisms.
- Explain the concept of an attention mechanism as originally introduced for encoder-decoder RNN models (Bahdanau/Luong-style additive and multiplicative attention), and formally define how an alignment score between a decoder state and encoder states is computed and converted into a context vector.
- Derive the scaled dot-product attention formula, $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$, explaining precisely the role and dimensionality of the Query, Key, and Value matrices, and mathematically justify why the $1/\sqrt{d_k}$ scaling term is necessary to stabilize gradients through the softmax.
- Explain, with a worked numerical example on a short toy sequence (3–4 tokens, small hand-chosen Q/K/V values), how attention weights are computed and how they determine the resulting context vector for one query token.

Part B – Transformer Architecture (25 Marks)

- Formally define multi-head attention: explain why multiple attention heads are used instead of a single attention computation, how the Q/K/V projections are split and later concatenated, and what kind of complementary relational patterns different heads are empirically observed to capture.
- Explain why Transformers, unlike RNNs, have no inherent notion of token order, and derive the sinusoidal positional encoding formulation, explaining why sinusoidal functions of varying frequency were chosen and what property (relative position representability) this design enables. Contrast this with learned positional embeddings.
- Diagram and explain the full encoder block (multi-head self-attention, residual connections, layer normalization, position-wise feed-forward network) and the full decoder block (masked self-attention, encoder-decoder cross-attention, feed-forward network), explaining the specific purpose of causal masking in the decoder's self-attention.

- Explain the role of residual connections and layer normalization in enabling stable training of deep Transformer stacks, connecting this explanation to the vanishing-gradient discussion from the RNN/LSTM tasks.
- Analyze the computational complexity of self-attention ($O(n^2 \cdot d)$ in sequence length n) versus recurrent computation ($O(n \cdot d^2)$), and discuss the practical implications of this trade-off for long-sequence modeling, referencing at least one documented approach to mitigating quadratic attention cost (e.g., sparse attention, linear attention, sliding-window attention) without deriving it in full.

Part C – Applied Transformer Analysis (35 Marks)

- Select a publicly available pretrained Transformer-based model (e.g., BERT, RoBERTa, DistilBERT, or a comparable encoder model) accessible through an established library, and justify the selection with reference to the model's architecture and intended use case.
- Using the pretrained model as-is (no architectural modification and no training from scratch), apply it to a text classification or sequence understanding task via fine-tuning of the existing model or direct inference, and describe the exact pipeline used: tokenization scheme (e.g., WordPiece/BytePair), input formatting, and any classification head used on top of the pretrained encoder.
- Extract and visualize the attention weights from at least one self-attention head for a sample input sequence, and interpret what linguistic or relational pattern (if any) the visualized head appears to be attending to.
- Evaluate the applied model's performance using accuracy, precision, recall, F1-score, and a confusion matrix on a held-out test set, and compare these results against the LSTM-based text classification results obtained in Task 25 on a comparable or identical dataset.
- Discuss, with reasoning grounded in the Part A and Part B derivations, why the Transformer-based model's performance and training characteristics differ from the LSTM model's (e.g., convergence speed, ability to capture long-range dependencies, sensitivity to sequence length).

Part D – Analysis & Documentation (20 Marks)

- Summarize the full conceptual pipeline: attention formulation, multi-head structure, positional encoding strategy, and encoder/decoder composition, explaining the reasoning behind each component's inclusion in the architecture.
- Summarize the quantitative findings from the Part C evaluation and the LSTM-vs-Transformer comparison.
- Discuss at least two non-trivial conceptual challenges encountered while reasoning through the attention/Transformer derivations or while interpreting the extracted attention weights, and how each was resolved.
- Reflect critically on the limitations of the standard Transformer architecture (e.g., quadratic attention cost, lack of inherent recurrence/inductive bias for sequence order, data and compute requirements) and identify at least one architectural variant or optimization developed to address a specific limitation, without deriving it in full.

Assessment Breakdown

Assessment Area	Marks
From Recurrence to Attention	20
Transformer Architecture	25
Applied Transformer Analysis	35
Analysis & Documentation	20
Total	100

Note: This task is conceptual by design. Interns are not required to implement scaled dot-product attention, multi-head attention, or any Transformer block from scratch (e.g., in raw NumPy). All derivations in Parts A and B must be shown mathematically on paper/in the document, but code is only required in Part C, where an existing pretrained model must be used via an established library rather than built manually.